

A Comparative Analysis of Single-Task and Multi-Task Learning Approaches for a House Price Prediction Model

CSCI323 Group Project

Group: FT20

Topic: House Price Prediction, Project 5

A Comparative Analysis of Single-Task and Multi-Task Learning Approaches for a House Price Prediction Model	1
1 - Introduction	2
2 - Background Theory	3
2.1 Ridge Regression (STL)	3
2.2 Random Forest (STL)	3
2.3 Multi-Task Learning (MTL)	3
2.4 Insights from Referenced Research Paper (Gao et al., 2019)	4
3 - Solutions, Evaluation, and Discussions	4
3.1 Data Preprocessing	4
3.2 Ridge Regression (STL Ridge)	5
3.3 Random Forest (STL RF)	6
3.4 Multi-Task Learning (MTL $L_{2,1}$)	7
3.5 Evaluation Methodology	9
3.6 Final Results	10
3.7 Discussion - Result Analysis	10
4 - Conclusion	11
5 - Acknowledgement	12
6 - References	12
7 - Appendix	14

1 - Introduction

The objective of this project is to investigate how model performance differs with different task learning approaches. Specifically, we are comparing Single-Task Learning (STL) versus Multi-Task Learning (MTL) approaches for a House Price Prediction problem using the King County dataset from Kaggle (harlfoxem, 2016).

Our implementation aims to reproduce the MTL framework proposed in the reference paper (Gao et al., 2019) and to compare its performance against STL models. By doing so, we aim to examine whether MTL is truly more effective for house price prediction, as concluded in the paper.

House price prediction is a complex but highly relevant problem. The stakeholders, from real-estate professionals to ordinary individuals, who may become home buyers or sellers, face the need to understand price dynamics. Although social and economic factors influence property value, this report focuses solely on the intrinsic attributes of houses (structural features, geographic location, etc.). Even with these limited attributes, the house price prediction task is far from simple. For example, properties located near rivers may maintain value more steadily over time, and houses with the same floor area may still differ greatly in price depending on their construction year or geographical location.

In early studies, traditional regression models were commonly used for house price prediction, which attempted to model the entire housing dataset with a single predictive model, but performance deteriorated as datasets grew larger and more heterogeneous. Subsequent research divided the dataset into smaller segments and modelled each segment independently. However, these approaches failed to capture complex relationships across segments. They are limited by the assumption that the influence of each feature on the price remains constant across all houses. In reality, this assumption does not hold as the impact of each feature varies depending on multiple factors, including region, environment, and property type, among others.

To overcome these limitations, the reference paper (Gao et al., 2019) introduced a MTL framework. MTL predicts house prices separately across regions or feature-based clusters, enabling the model to learn localised relationships. The paper emphasised two key aspects: how tasks should be defined, and how relationships between tasks should be modelled. It demonstrated that task definition has a particularly strong impact on model predictive performance.

Inspired by this work, our project aims to reproduce the MTL framework and compares it with two other STL approaches, one linear and one non-linear. Ridge Regression was selected as the stable and simple linear model, while Random Forest was chosen as the non-linear baseline because it achieved strong and consistent performance in the reference study.

Our implementation uses the House Sales in King County, USA dataset (harlfoxem, 2016), which contains over 21,000 housing records. The dataset includes diverse structural and spatial features such as house

condition, grade, latitude, and longitude, making it highly suitable for our goal of comparing house price prediction models.

The proposed approach has broader potential beyond real estate analytics. Identifying the most influential features across regions can support urban planning and local development policies. Since property prices are closely tied to investment value, this research can also contribute to data-driven real estate decision-making. Moreover, because the dataset contains both spatial and structural factors, it can be extended to personalised property recommendation systems.

2 - Background Theory

2.1 Ridge Regression (STL)

Ridge Regression is based on the theory that a linear model can be improved by controlling the magnitude of its coefficients. It introduces a L_2 regularisation term that penalises large coefficients and shrinks them towards zero (Friedrich et al., 2022), hence minimising variance and improving stability. The model is simple, stable and interpretable, however it struggles to capture non-linear relationships between features which are recurrent in housing markets. Ridge is suitable as a baseline machine learning model as standard linear regression minimises the sum of squared errors (SSE), and many housing features such as square footage, condition and number of bedrooms are strongly correlated with housing price.

2.2 Random Forest (STL)

Random Forest is an ensemble learning technique introduced by Breiman (2001). It extends decision-tree learning by implementing two sources of randomness: bootstrap sampling (bagging) of the data and random selection of features at each split. These methods reduce correlation between trees, which is important because the generalisation error depends on both the strength (“accuracy”) of individual trees and the degree of correlation among them (Breiman, 2001). After training, the predictions of all trees are aggregated, typically by averaging in regression tasks. This results in a model that captures non-linear relationships while maintaining low variance. Due to its flexibility and minimal preprocessing requirements, Random Forest is widely used as a strong baseline in machine learning applications.

2.3 Multi-Task Learning (MTL)

Multi-task learning (MTL) is an approach to inductive transfer that improves generalization by learning multiple related tasks in parallel using a shared representation (Caruana, 1997). In this framework, shared layers extract general patterns from input data that are useful across tasks. Because tasks are learned jointly rather than independently, whatever that is learned for each task serves as an inductive bias for other tasks, helping the model generalize better (Caruana, 1997). Recent research has shown that joint learning enables the model to make use of correlations among tasks, where knowledge gained from one task can improve learning in others through inductive transfer (Ruder, 2017). This shared learning helps to reduce

overfitting, improve data efficiency, and lead to more stable training, especially when individual tasks have limited or imbalanced data. However, MTL is most effective when tasks are closely related. Sharing information with unrelated tasks can lead to negative transfer, where performance degrades rather than improves (Ruder, 2017). These principles demonstrate how MTL utilises task relationships to improve overall performance, which would be a good baseline for our implementation.

2.4 Insights from Referenced Research Paper (Gao et al., 2019)

The referenced research paper (Gao et al., 2019), examines the house price prediction problem through a MTL framework and highlights how the MTL task structure and feature profiles influence model performance through shared learning. The study defined a total of four location-centred feature profiles, housing (consisting of structural and regional features), education, transportation and facility profiles, each capturing a different aspect of the factors that affect a property's value.

The authors examined three MTL-based approaches: MTL with L_1 -norm regularisation, $L_{2,1}$ -norm regularisation and graph regularisation. Among the three, MTL with graph regularisation had the strongest performance across the different task definitions. The research paper also reports that, in general, performance of MTL-based models surpasses that of STL approaches provided that the tasks are meaningfully separated and share underlying relationships.

For our implementation, we chose to adopt the MTL $L_{2,1}$ regularised framework for the MTL baseline instead of the graph regularised MTL to strike a balance between showcasing the capabilities of an MTL and computational feasibility. This aligns with the baseline comparison established in the reference paper and is adapted for the characteristics and scale of our selected dataset.

3 - Solutions, Evaluation, and Discussions

3.1 Data Preprocessing

For our implementation, we used the House Sales in King County, USA dataset (harlfoxem, 2016). Our data preprocessing pipeline consists of, in order, loading raw data, exploratory data analysis, data cleaning, feature engineering, log transformation, train-test split and scaling. The steps mentioned in this section were applied to Ridge Regression and Multi-Task Learning models, whilst the Random Forest model used the unscaled train-test split. Model-specific preprocessing (i.e. MTL task partitioning) is discussed later in the corresponding model section. All models were trained on log-transformed price values, with evaluation performed in both log and actual spaces but the latter is used in our final evaluation as it is more interpretable with real price values.

The dataset did not contain any missing values, therefore no imputation was needed. However, it contained some entries which we deemed invalid based on realistic housing constraints, including rows with 0 bathrooms and bedrooms which were unrealistic, and even a row of data with an extreme outlier of 33

bedrooms (Figure 3.1a). Hence, we dropped such rows from the dataset, ensuring that these samples did not distort the model behaviour. Apart from unrealistic entries, we inspected numerical outliers (e.g. unusually large living area or lot size) but retained them as they appeared to be valid rare properties rather than erroneous entries.

Before further preprocessing, we inspected the data using histograms (Figure 3.1a - 3.1c), scatter plots (Figure 3.1d - 3.1f) and a correlation matrix (Figure 3.1g) to understand the feature distributions, identify skewed features and observe relationships between features such as the dependency between price and key attributes.

To provide the models with more meaningful inputs, several additional features were created from existing columns:

- House age: a feature that was computed as the difference between the transaction year (extracted from the 'date' feature) and the year built
- Renovation flag: binary feature indicating whether the house had been renovated
- Basement flag: binary feature derived from the 'sqft_basement' feature to reflect if the house had a basement
- Cyclic month encoding: a temporal feature extracted from the 'date' feature and encoded using sine-cosine transformations to preserve its periodic structure

We removed redundant features and less informative features such as transaction ID and transaction date, along with intermediate columns used for feature engineering.

The generated histograms (Figure 3.1a - 3.1c) showed that several numeric features, including the target variable (price) exhibited right-skewed distributions. Log transformation was applied to these features to reduce skewness. Continuous features were scaled using StandardScaler, while binary, cyclic and spatial cluster features were intentionally left unscaled to retain their inherent meaning. Finally, the dataset was then split into a 70/30 train-test split, ensuring a consistent evaluation setup across all three baseline models.

3.2 Ridge Regression (STL Ridge)

Ridge Regression is a type of linear regression technique that calculates the minimum sum of squared vertical distances between the data points and the prediction line (Murel & Kavlakoglu, n.d.).

Formula for Ridge Regression : $\sum_{i=1}^n (Y_i - \hat{Y}_i)^2 + \lambda \sum_{j=1}^p B_j^2$ (Murel & Kavlakoglu, n.d.)

Formula for Residual sum of squares: $\sum_{i=1}^n (Y_i - \hat{Y}_i)^2$ and the Penalty Term: $\lambda \sum_{j=1}^p B_j^2$

The first term $\sum_{i=1}^n (Y_i - \hat{Y}_i)^2$ is identical to the Least Square regression method, and it represents the residual sum of squares. The second term $\sum_{j=1}^p B_j^2$ is an additional L_2 penalty, which discourages large coefficient values and shrinks them toward zero. The λ controls the balance between the two terms and helps prevent overfitting. As λ increases, the coefficients shrink further, resulting in a simpler and more stable model (Murel & Kavlakoglu, n.d.).

Based on the two graphs (Starmer, 2018), showing the effect of varying lambda on the ridge regression line (Figure 3.2a - 3.2b), as the penalty value increases, the prediction line will be adjusted to not overfit the data. However, as the penalty value substantially increases, the prediction line becomes more and more horizontal and the model will have a higher chance of underfitting.

We selected Ridge Regression for one of our models for the project as it is computationally efficient, handles multicollinearity and overfitting by adding a penalty value (Murel & Kavlakoglu, n.d.). It also improves model stability by constraining the size of coefficients and has the ability to perform better with many features.

However, we noted that as a linear model, Ridge is unable to capture nonlinear relationships unless such features are explicitly engineered. Ridge does not perform feature selection, since its penalty shrinks coefficients without setting them exactly to zero.

Training Setup:

```
# Ridge Regression Model

# Initialize and train the Ridge model
ridge_model = Ridge(alpha=1.0) # You can tune the alpha parameter
ridge_model.fit(X_train_ridege, y_train_ridege)
```

Above is our training set up for Ridge Regression, using penalty value = 1. For Ridge Regression, the features are scaled to have zero mean and unit variance using StandardScaler. This is necessary because the L_2 penalty used by Ridge is highly sensitive to the magnitude of the coefficients. If the features have different scales, the coefficient estimates may become distorted and the penalty will not be applied fairly. Scaling the features ensures that the regularisation affects all coefficients consistently.

3.3 Random Forest (STL RF)

Random forest is an ensemble machine learning model that combines multiple decision trees (Figure 3.3a). Each tree in the forest is trained on a random sample of the data and considers only a random subset of features when making splits. The Random Forest model was chosen for its robustness against overfitting and ability to handle non-linear relationships. The model is robust against noise as Random Forest combines the forecasts of several decision trees. However, we have to be careful not to use too many trees in the forest or train a Random Forest on a large dataset, as doing so can be computationally expensive. A large amount of computing power would be needed to aggregate the predictions since each tree is trained

separately. This causes greater memory utilisation and training times, especially on systems with limited resources.

Formula for RF model: $\hat{y} = \frac{1}{B} \sum_{k=1}^B \hat{y}(k)$, where B represents the number of trees and $\hat{y}(k)$ represents the prediction of tree B for input k (B., 2025).

Training Setup:

```
# Random Forest model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_rf, y_train_rf)
```

For our Random Forest model, we decided on $n_estimators=100$, where the model builds 100 decision trees, and each tree is trained on a different subset of the data. The final prediction is computed from the average of all trees' predictions and its values will then be used for the regression metrics.

The Random Forest model uses unscaled data for the regression metrics. This is because Random Forest does not require feature scaling because decision trees within the model split data based on threshold comparisons rather than distances.

3.4 Multi-Task Learning (MTL $L_{2,1}$)

MTL Task Partitioning Logic (Additional preprocessing for MTL only):

To enable the $L_{2,1}$ -regularised MTL model to learn both shared and task-specific patterns, the dataset was partitioned into multiple related tasks. Since the King County dataset (harlfoxem, 2016) lacks the specific education, transportation and facility features and profiles used in the reference paper (Gao et al., 2019), we adopted a spatial-structural partitioning that mirrors the paper's design as much as possible and adapted it to our smaller dataset and available features.

Spatial Partitioning:

K-means clustering was applied to geographical coordinates ('lat' and 'long' features) to group the dataset into meaningful region clusters. After testing values of k from 3 to 10 and examining both region separation (Figure 3.4a) and cluster size distributions (Figure 3.4b), we selected $k = 4$, as it yielded the most balanced and meaningful clusters (Figure 3.4c).

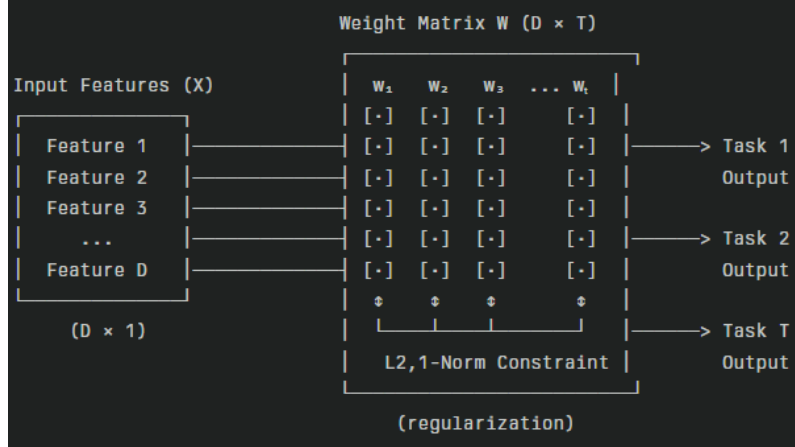
Structural Partitioning:

The `sqft_living` feature was binned into two quantile-based groups to capture differences between smaller and larger homes.

Spatial-Structural Partitioning:

By combining the four region clusters and two house size bins, we defined a total of eight tasks. Each sample was assigned a task ID based on its (*region_cluster* $\times$ *sqft_living_bin*) combination. This task structure allows the MTL model to learn and capture region-specific and size-specific behaviour through task-specific outputs, whilst still sharing information across similar tasks. Task distribution plots and data counts per task (Figure 3.4d) are provided in the appendix.

Our Architecture:



Our MTL $L_{2,1}$ approach adopts the linear regression framework as proposed by the research paper (Gao et al., 2019). For each task p , the linear function is defined as $f_p(x_p) = x_p w_p$, with all task-specific weight vectors w_p stacked into a weight matrix $W \in \mathbb{R}^{DT}$, where D is the number of input features and T is the number of tasks (Gao et al., 2019). Unlike typical MTL models that incorporate shared hidden layers (Figures 3.4e - 3.4f), our architecture consists solely of the weight matrix W , with each column representing a task-specific weight vector and each row representing an input feature, as seen from the figure above. Each task is associated with a specific column of W , and predictions for a given task are produced by applying the corresponding linear weight vector directly to the input features.

$L_{2,1}$ Regularisation and Soft Parameter Sharing

During training, the model does not treat these task-specific weight vectors independently. Instead, $L_{2,1}$ regularisation is applied across the rows of W . This regularisation computes the L_2 norm of each feature's weights across all tasks (i.e. across a row) and then sums these values across all features, resulting in group sparsity. This encourages the model to select a common subset of relevant input features across all tasks whilst still allowing for each task to be distinct, and irrelevant input features are suppressed. This results in soft parameter sharing. The model is trained by minimising the following objective function:

$$\arg \min_W \sum_{p=1}^P \left\| x_p w_p - y_p \right\|_F^2 + \theta_1 \|W\|_{2,1}$$

as formulated in Equation 3 (Gao et al., 2019, p. 7).

Training Setup

The MTL model was implemented in PyTorch and trained directly on the weight matrix W , since the architecture contains no hidden layers.

The Adam optimiser was used with a learning rate of 0.03 and $\varepsilon = 10^{-8}$. The weights were initialised using a fixed seed (42) with small Gaussian noise (standard deviation = 0.01) to ensure stable and reproducible optimisation.

During each step, the model first performs a forward pass by selecting the appropriate task-specific weight vector from the weight matrix W based on the task ID and computes predictions using the linear function $f_p(x_p) = x_p w_p$.

The loss function is the combination the mean squared error (MSE) with the $L_{2,1}$ regularisation term (Ω): $L(W) + \Omega(W)$, where $L(W) = \sum_{p=1}^P \left\| x_p w_p - y_p \right\|_F^2$ and $\Omega = \theta_1 \|W\|_{2,1}$ (Gao et al., 2019, pp. 6-7). This encourages group sparsity across the rows of W .

After computing the loss, backpropagation is applied to compute gradients of the loss with respect to W , and the Adam optimiser updates the parameters accordingly. This lets the model learn from both the prediction errors and the $L_{2,1}$ regularisation at the same time.

Early stopping was applied by monitoring the loss during optimisation. Training was stopped if no improvement was observed for 250 iterations and the best-performing weight matrix W was restored at the end of training.

For hyperparameter selection, a 5-fold cross-validation was used to evaluate different values of the $L_{2,1}$ regularisation strength, λ (refers to θ in the equation). The value that yielded the highest mean validation R^2 score was selected for final training, which was $5 * 10^{-4}$ (0.0005).

Using the best λ , the final MTL model was trained on the full training set for up to 3500 steps, producing the final weight matrix used for task-specific predictions.

3.5 Evaluation Methodology

To ensure consistency with the evaluation metrics used in the referenced research paper (Gao et al., 2019), we used Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE), as these metrics capture the absolute and squared prediction errors. We also reported R^2 as an additional metric as it is a commonly used indicator of how well a model explains variance in regression tasks.:

- RMSE is the square root of the average of squared differences between predicted and actual values, and penalises large errors heavily. In general, a smaller RMSE value would indicate higher model accuracy

- MAE is the average absolute difference between predicted and actual values, where a lower value is an indicator of better performance as well
- R^2 is the proportion of variance that is explained by the model, and is often in the range: $0 \leq R^2 \leq 1$, where a value of 1 would indicate perfect prediction and 0 would indicate that the model does no better than predicting the mean. A value less than 0 indicates that the model performs worse than the mean baseline, hence a value closer to 1 would indicate a better performing model

All three models were trained and evaluated in log-price space, and after prediction, the outputs were converted back into real-price units and the evaluation metrics were computed in both scales. However, for the final result comparison presented, we use only the real-price metrics, to give an idea of how far off the predictions are in actual price values, offering a more interpretable evaluation of model performance.

3.6 Final Results

	Model	R^2	RMSE (\$)	MAE (\$)
0	Ridge Regression	0.7345	190,739	111,948
1	Random Forest	0.8721	132,398	70,032
2	MTL ($L_{2,1}$)	0.8111	160,915	95,423

The final model comparison in real price units is shown in the figure above. Among the three baseline models implemented, the STL RF model achieved the highest R^2 value and lowest errors (RMSE and MAE), making it the strongest overall performer, followed by the MTL $L_{2,1}$ and then the STL Ridge models respectively.

3.7 Discussion - Result Analysis

The STL Random Forest (RF) model demonstrated the strongest performance among the three baselines. Its ability to handle non-linear feature interactions and model the complex relationships in the dataset explains its high R^2 value and low error scores. In contrast, the STL Ridge Regression model captured only linear trends in the transformed data and therefore struggled with more complex housing price patterns, resulting in the lowest performance. Our MTL $L_{2,1}$ model was able to successfully learn some *region-size* patterns and exhibited some task-specific behaviour. However, as a linear framework, its performance was unable to surpass the flexibility of the RF model.

A key reason for this difference in performance becomes clear when comparing our implementation setup with the research paper (Gao et al., 2019). In general, the paper’s MTL baselines outperformed all other STL models because it was able to leverage on the much larger dataset (136,394 house transaction records) that contained four rich location-driven feature profiles (house, education, transportation and facility profiles), with a total of 16 features across these feature profiles (e.g. for the education profile alone, there

were three features, school district, nearest schools and school rankings). These profiles produced semantically meaningful and highly related tasks, such as grouping the houses by school district quality or proximity to community facilities, among others. Such task definitions strengthened the shared learning in their MTL.

In contrast, the dataset we used (harlfoxem, 2016) contained only 21 columns and 21,613 rows, with only mostly structural house features, without the variety of contextual geographic features. As mentioned earlier, to remain consistent with the spatial-structural partitioning logic, we defined eight tasks using the region clusters and house size bins, but these tasks were less meaningful. With fewer and less diverse features in our dataset to contribute to our MTL task structure, and the fact that our MTL does not make use of hidden shared layers in its architecture, we were unable to exploit the shared-learning advantage of the MTL $L_{2,1}$ framework as effectively, compared to the implementation (of the MTL $L_{2,1}$ model) in the research paper (Gao et al., 2019), hence leading to weaker MTL performance, relative to the RF model.

4 - Conclusion

From the results of our implementation, we concluded that data richness and quality are equally as important as model complexity. Additionally, while MTL will excel when the tasks are distinct yet related, as demonstrated in the research paper, our implementation shows that its advantages diminish when the dataset has limited variation, both in spatial or contextual features. As a result, the non-linear STL RF model outperformed the MTL and Ridge models on our simpler dataset.

Our project also highlights that traditional non-linear models like the STL RF model can outperform advanced frameworks like MTL when the features available for model training do not support complex shared-task structures.

For future works, ensuring that the dataset has richer spatial and contextual features, such as school zones, transport accessibility, etc., would improve task definition and structure to harvest the full potential of an MTL model. Another possible direction to explore to expand the learning on this topic would be to implement neural network-based or graph regularised MTL, which was mentioned as the best performing model in the research paper (Gao et al., 2019).

Our key takeaway from this project and topic is that the effectiveness and potential of a MTL model largely depends on the richness of the dataset and meaningful task definitions. It is only when these two interrelated conditions are met that MTL can outperform STL approaches. Otherwise, without either of these interconnected factors, simpler models may perform better.

5 - Acknowledgement

We also acknowledge that generative AI tools (e.g. ChatGPT, Perplexity) were used as supplementary aid during the development of the MTL model. Since the research paper referenced did not provide an official codebase, we relied on its mathematical formulation and used generative AI only to help clarify how certain components (such as the $L_{2,1}$ norm, task-specific weight selection, optimisation loop, etc.) could be implemented in practice.

The final MTL implementation was written, adapted, and refined by our group based on our own understanding of the paper. No code was copied directly without modification. AI-generated examples served only as conceptual guidance, which we restructured to fit our dataset, architecture design, and training procedure.

6 - References

References

Ansa. (2025, January 20). *Mathematics Behind Random Forest Algorithm*. Medium.

<https://ai.plainenglish.io/mathematics-behind-random-forest-algorithm-c3b818597e0b>

Bhakta, S. S. [susmit_sekhar_bhakta] (2025, October 31). *Random Forest Algorithm in Machine Learning*. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>

Caruana, R. (1997, July). Multitask Learning. *Machine Learning*, 28(1), 41-75.

<https://doi.org/10.1023/A:1007379606734>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

<https://doi.org/10.1023/A:1010933404324>

Friedrich, S., Groll, A., Ickstadt, K., Kneib, T., Pauly, M., Rahnenführer, J., & Friede, T. (2022). Regularization approaches in clinical biostatistics: A review of methods and their applications. *Statistical Methods in Medical Research*, 32(2), 425–440. <https://doi.org/10.1177/09622802221133557>

Gao, G., Bao, Z., Cao, J., Qin, A. K., Sellis, T., Fellow, IEEE, & Wu, Z. (2019, January 7). Location-Centered House Price Prediction: A Multi-Task learning approach. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.1901.01774>

Sun, X., Panda, R., Feris, R., & Saenko, K. (2020, November 18). ADAShare: Learning What to Share for Efficient Deep Multi-Task Learning. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.1911.12423>

harlfoxem. (2016). *House Sales in King County, USA*. Kaggle.

<https://www.kaggle.com/datasets/harlfoxem/housesalesprediction>

Murel, J., & Kavlakoglu, E. (n.d). *What is ridge regression?* IBM.

<https://www.ibm.com/think/topics/ridge-regression>

Ruder, S. (2017, June 15). An Overview of Multi-Task Learning in Deep Neural Networks. *arXiv (Cornell University)*. <https://doi.org/10.48550/arXiv.1706.05098>

Srivastava, S. (2025, July 11). *Introduction to Multi-Task Learning(MTL) for Deep Learning*. GeeksforGeeks.

<https://www.geeksforgeeks.org/deep-learning/introduction-to-multi-task-learningmtl-for-deep-learning/>

Starmer, J. [StatQuest with Josh Starmer]. (2018, September 25). *Regularization Part 1: Ridge (L2)*

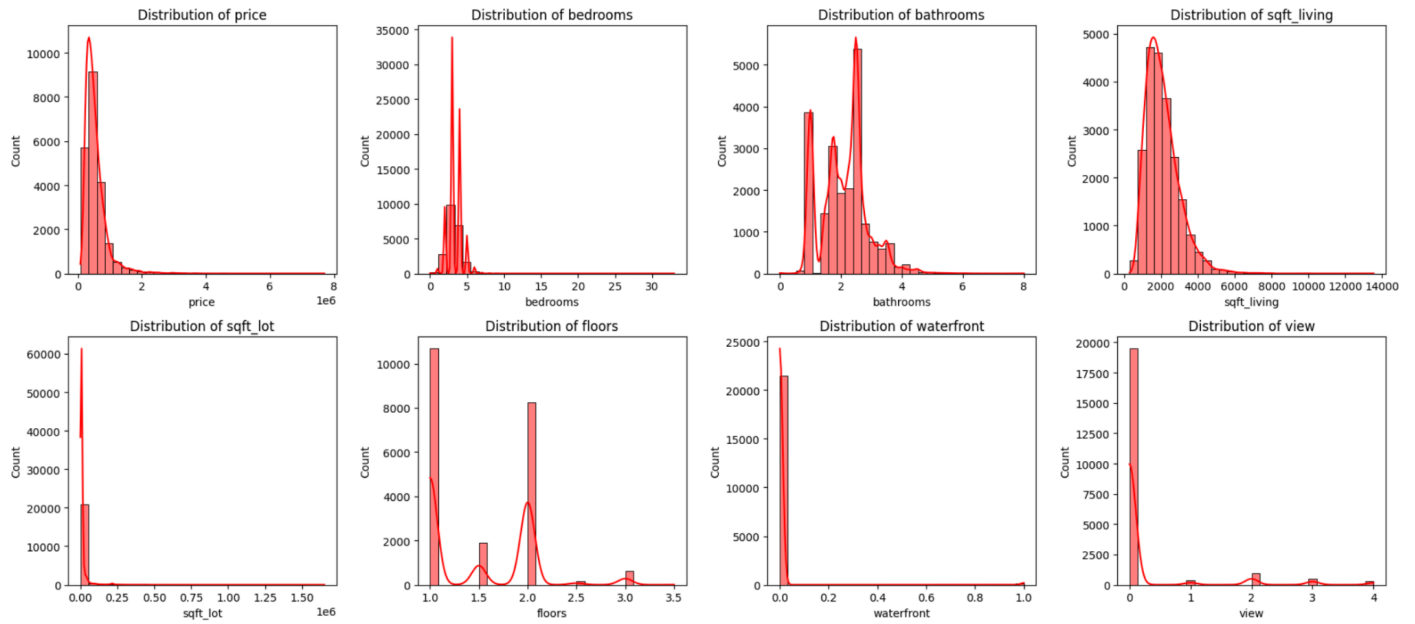
Regression. [Video] YouTube. <https://youtu.be/Q81RR3yKn30>

OpenAI. (2025, November 12). *ChatGPT (Version 5.1)* [Large language model]. <https://chat.openai.com/chat>

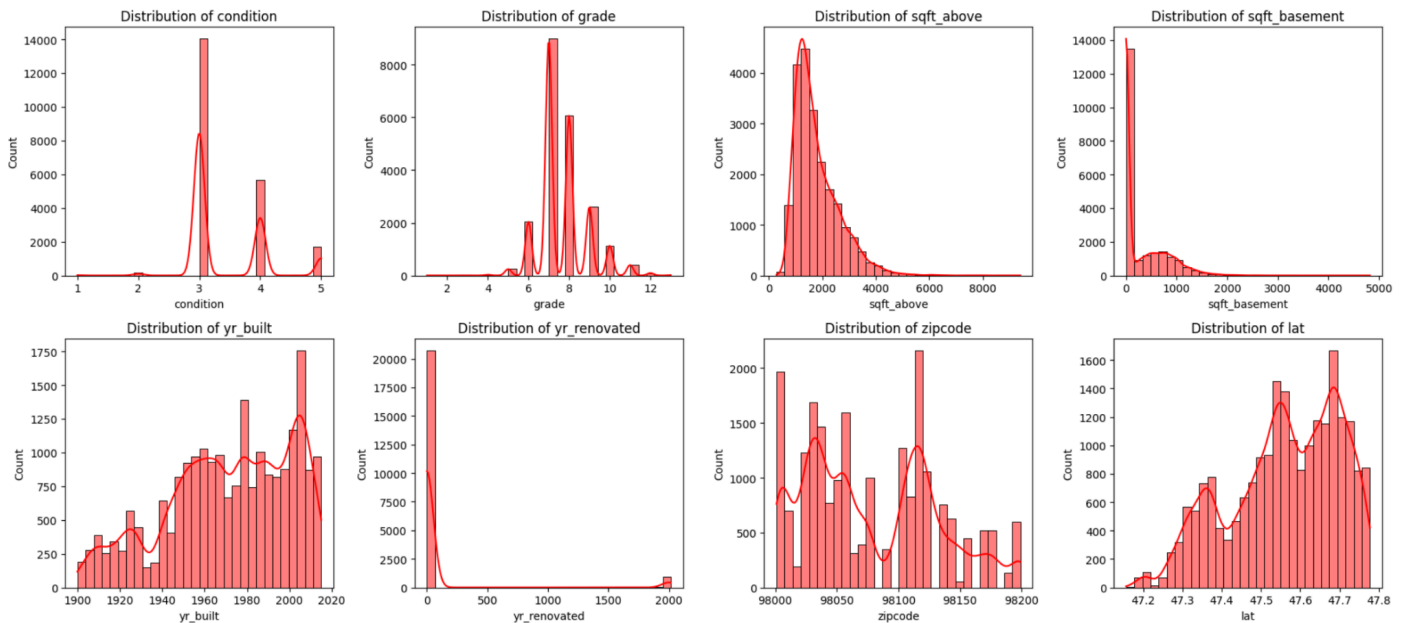
Perplexity AI. (2025, November 6). *Comet Assistant (November 6 release)* [Large language model].

<https://www.perplexity.ai/>

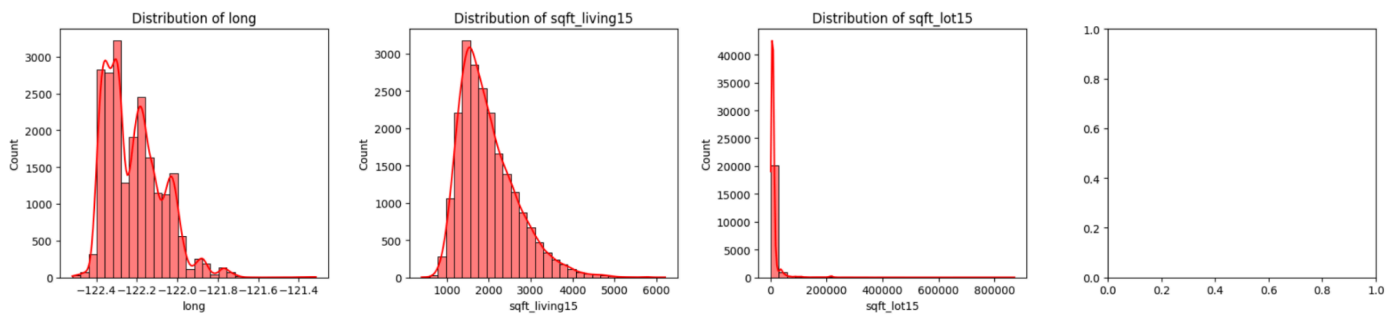
7 - Appendix



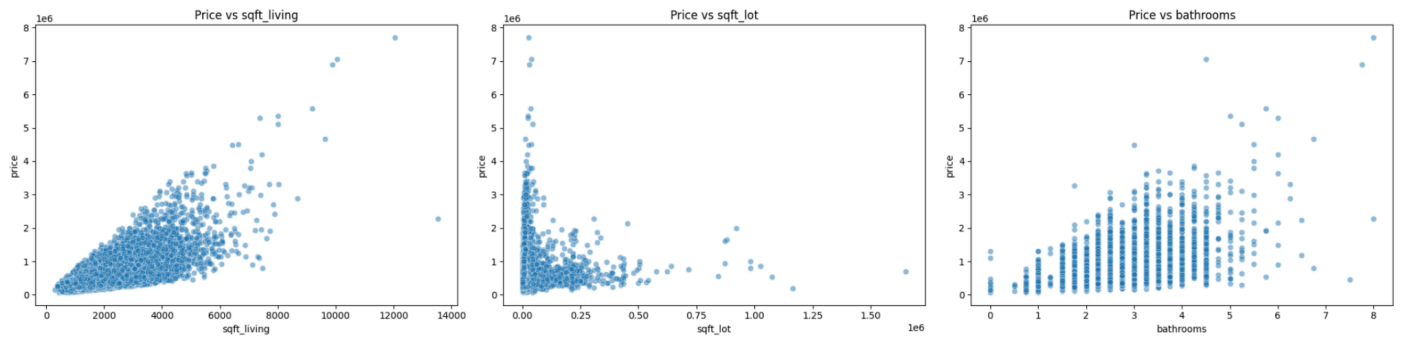
(Figure 3.1a) First 8 histograms plotted to visualise feature distribution



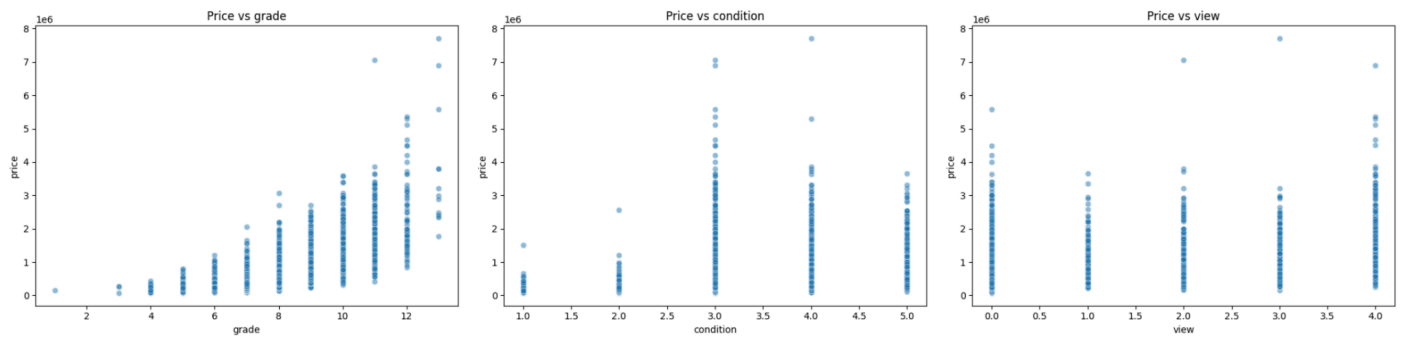
(Figure 3.1b) Next 8 histograms plotted to visualise feature distribution



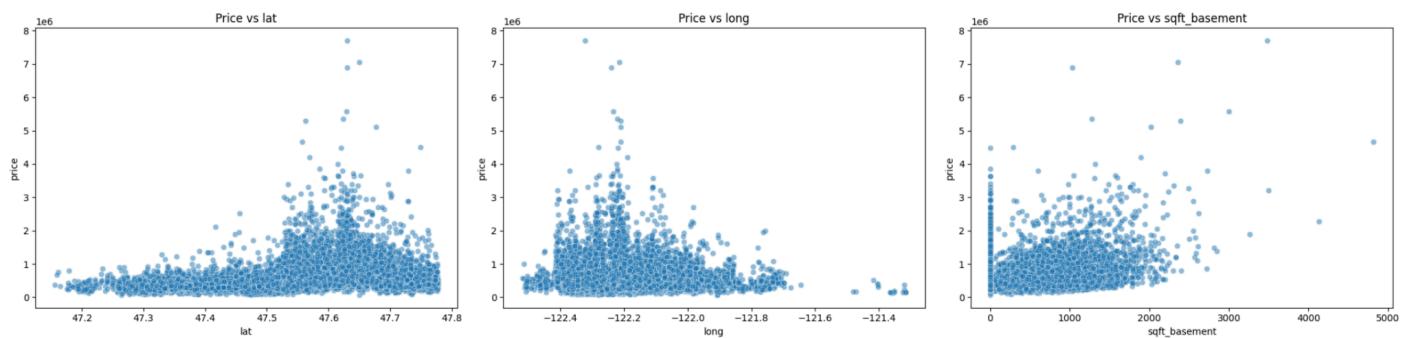
(Figure 3.1c) Final 3 histograms plotted to visualise feature distribution



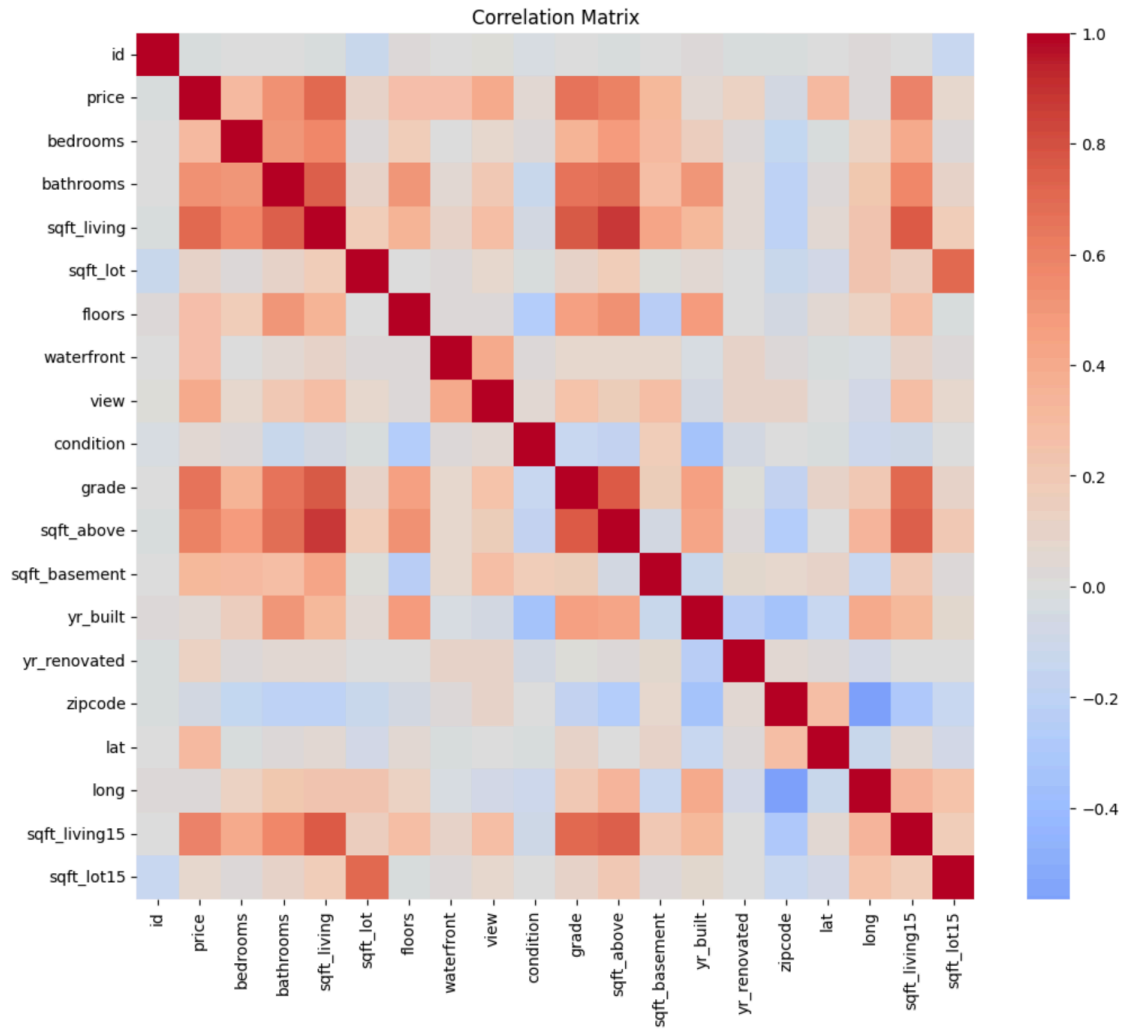
(Figure 3.1d) First row of histograms generated as part of exploratory data analysis



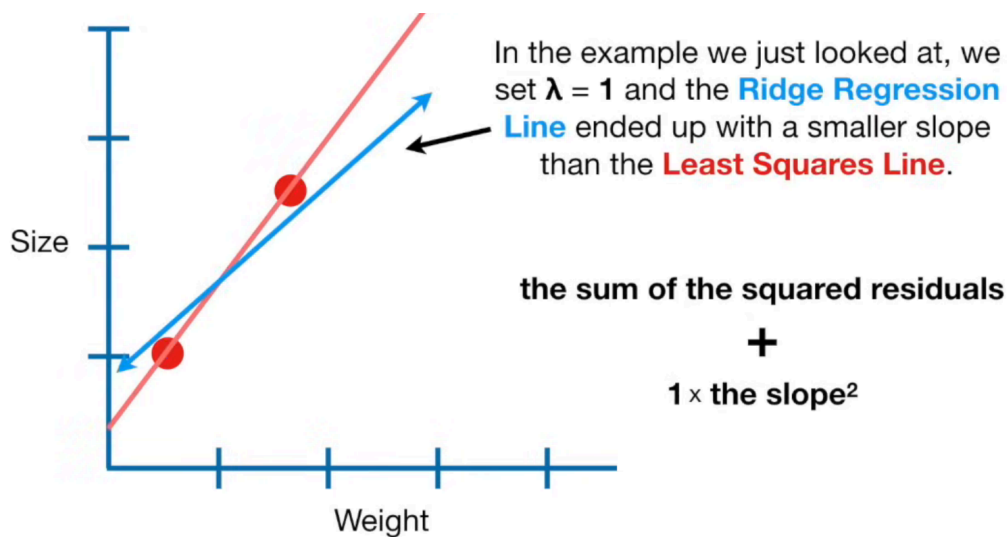
(Figure 3.1e) Second row of histograms generated as part of exploratory data analysis



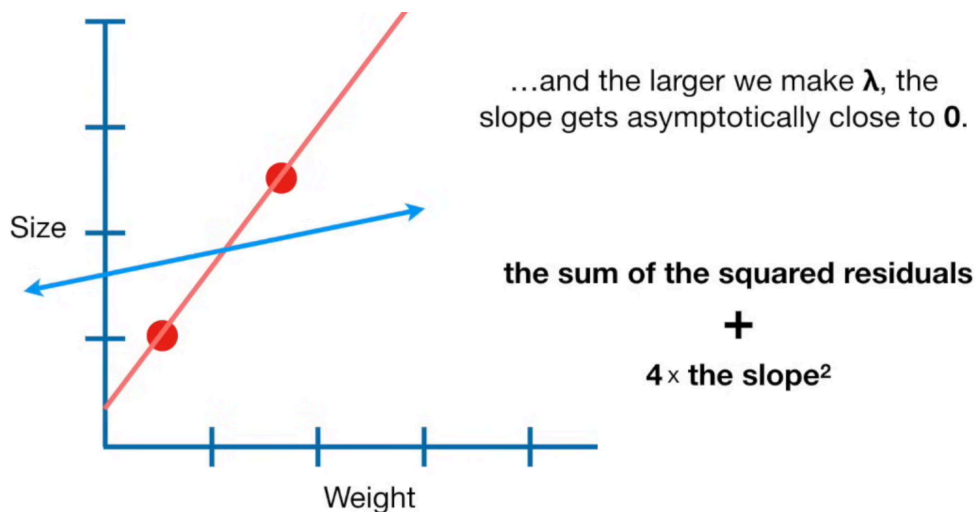
(Figure 3.1f) Last row of histograms generated as part of exploratory data analysis



(Figure 3.1g) Screenshot of correlation matrix



(Figure 3.2a) Screenshot Showing Effects of Lambda = 1 for Ridge Regression Model (Starmer, 2018)



(Figure 3.2b) Screenshot Showing Effects of Lambda = 4 for Ridge Regression Model (Starmer, 2018)

```

# Make predictions on the test set
y_pred_ride = ridge_model.predict(X_test_ride)

# Evaluate the model
rmse_ride = np.sqrt(mean_squared_error(y_test_ride, y_pred_ride))
mae_ride = mean_absolute_error(y_test_ride, y_pred_ride)
r2_ride = r2_score(y_test_ride, y_pred_ride)

# plot Ridge Regression actual vs predicted log price
plt.figure(figsize=(8,6))
plt.scatter(y_test_ride, y_pred_ride, alpha=0.5)
plt.plot([y_test_ride.min(), y_test_ride.max()],
         [y_test_ride.min(), y_test_ride.max()],
         'r--', lw=2, label="Ideal Fit (y=x)")
plt.xlabel("Actual Price (log-transformed)")
plt.ylabel("Predicted Price (log-transformed)")
plt.title("Ridge Regression: Actual vs Predicted Prices (Log Price Space)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

print("\nRidge Regression Model Evaluation (Log Price Space):")
print(f"R² : {r2_ride:.4f}")
print(f"RMSE : {rmse_ride:.4f}")
print(f"MAE : {mae_ride:.4f}")

```

(Figure 3.2c) code running and evaluating the Ridge model



(Figure 3.2d) Predicted vs Actual Prices (in Log Price Space) for Ridge Regression Model

```
Ridge Regression Model Evaluation (Log Price Space):
R² : 0.7630
RMSE : 0.4935
MAE : 0.3771
```

(Figure 3.2e) Evaluation Metrics (in Log Price Space) for Ridge Regression Model

```
# Convert scaled log(price) -> unscaled log(price)
y_test_ridge_unscaled = scaler_y.inverse_transform(y_test_ridge.reshape(-1,1)).ravel()
y_pred_ridge_unscaled = scaler_y.inverse_transform(y_pred_ridge.reshape(-1,1)).ravel()

# Convert unscaled log(price) -> real price in $
y_true_ridge_price = np.expml(y_test_ridge_unscaled)
y_pred_ridge_price = np.expml(y_pred_ridge_unscaled)

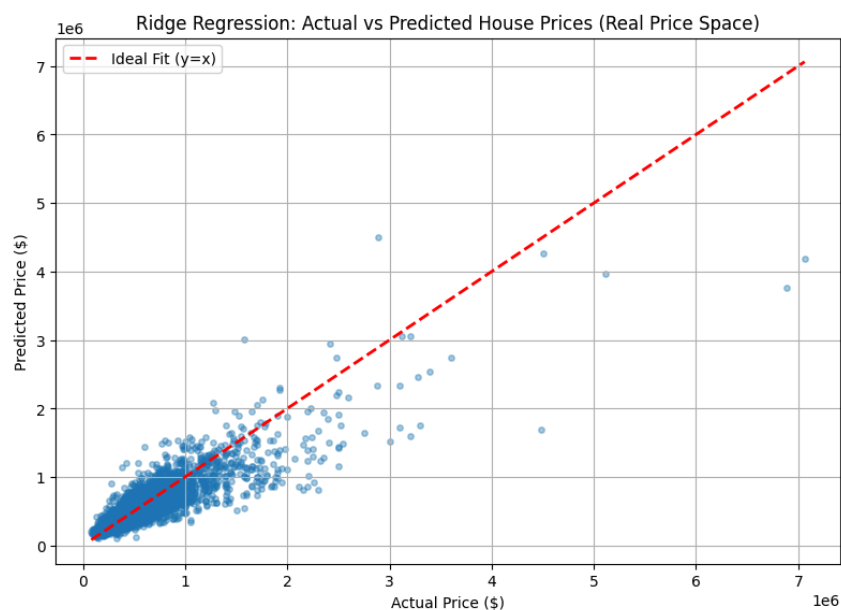
# Evaluate in real price units
r2_ridge = r2_score(y_true_ridge_price, y_pred_ridge_price)
rmse_ridge = np.sqrt(mean_squared_error(y_true_ridge_price, y_pred_ridge_price))
mae_ridge = mean_absolute_error(y_true_ridge_price, y_pred_ridge_price)

# Plot Ridge Regression actual vs predicted true price
plt.figure(figsize=(8, 6))
plt.scatter(y_true_ridge_price, y_pred_ridge_price, alpha=0.4, s=15)
plt.plot(
    [y_true_ridge_price.min(), y_true_ridge_price.max()],
    [y_true_ridge_price.min(), y_true_ridge_price.max()],
    'r--', lw=2, label="Ideal Fit (y=x)"
)

plt.xlabel("Actual Price ($)")
plt.ylabel("Predicted Price ($)")
plt.title("Ridge Regression: Actual vs Predicted House Prices (Real Price Space)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

print("\nRidge Regression Performance (Real Price Space)")
print(f"R² : {r2_ridge:.4f}")
print(f"RMSE : {rmse_ridge:.0f}")
print(f"MAE : {mae_ridge:.0f}")
```

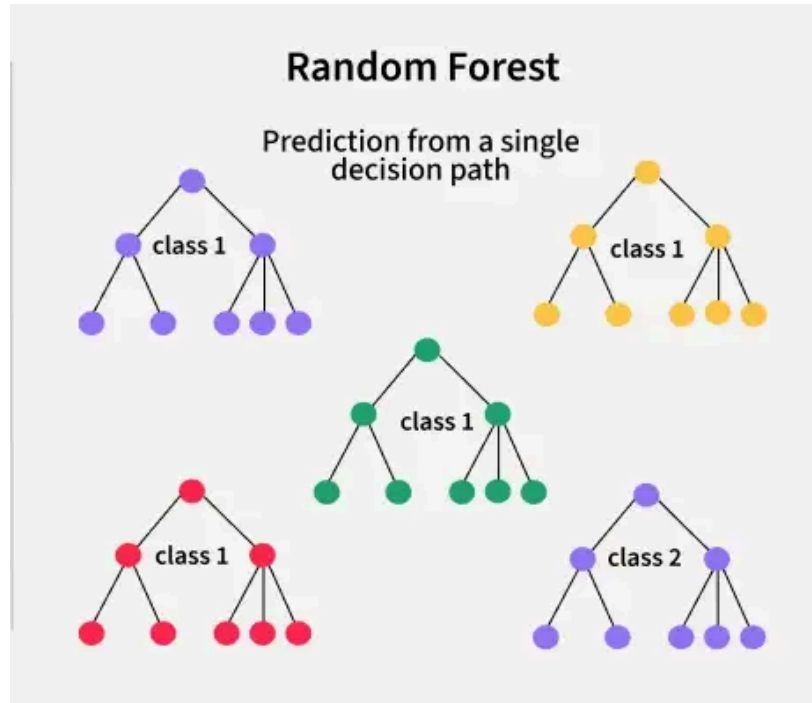
(Figure 3.2f) Converting log price to real price units



(Figure 3.2g) Predicted vs Actual Prices (in Real Price Space) for Ridge Regression Model

Ridge Regression Performance (Real Price Space)
 R^2 : 0.7345
RMSE : 190,739
MAE : 111,948

(Figure 3.2h) Evaluation Metrics (in Real Price Space) for Ridge Regression Model



(Figure 3.3a) Random Forest visualisation



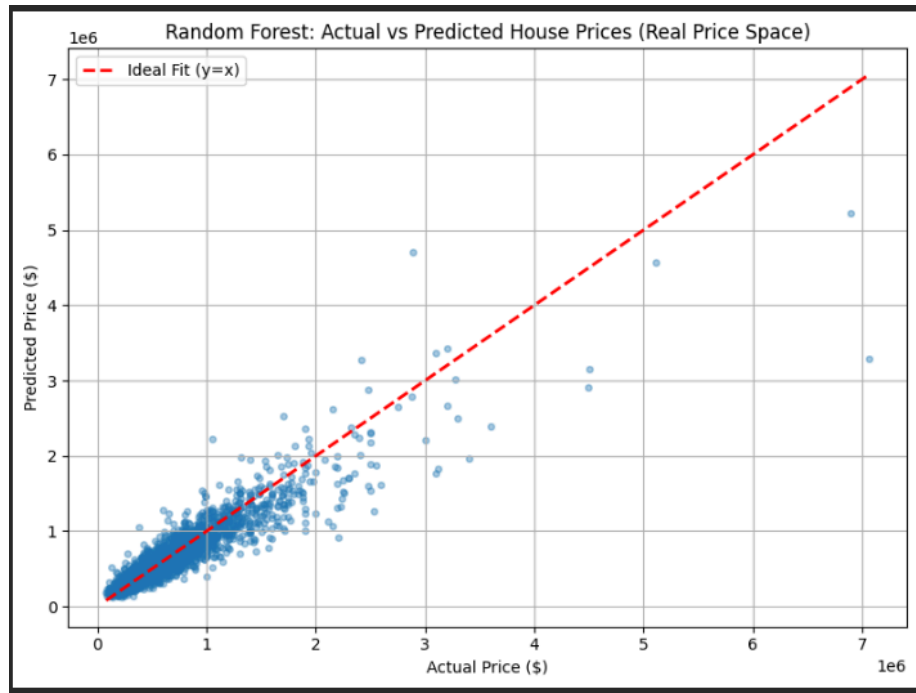
(Figure 3.3b) Predicted vs Actual Prices (in Log Price Space) for Random Forest Model

```

Random Forest Performance (Log Price Space)
R2 : 0.8862
RMSE : 0.1765
MAE : 0.1254

```

(Figure 3.3c) Evaluation Metrics (in Log Price Space) for Random Forest Model



(Figure 3.3d) Predicted vs Actual Prices (in Real Price Space) for Random Forest Model

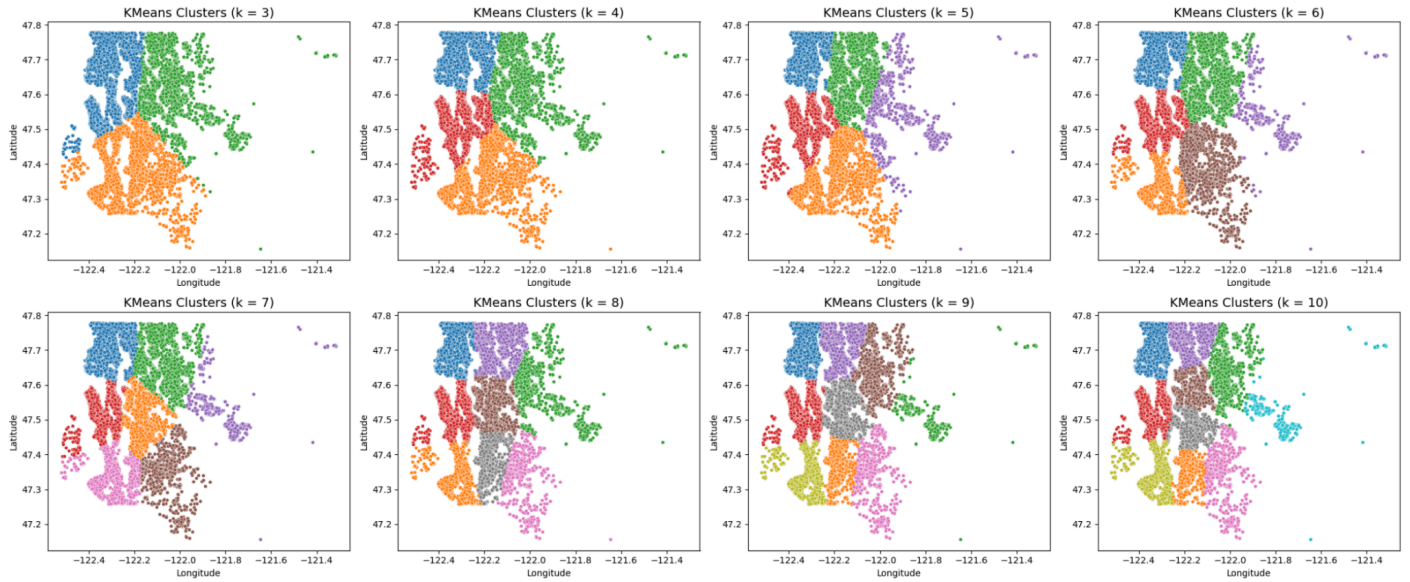
```

Random Forest Performance (Real Price Space)
R2 : 0.8610
RMSE : 135,696
MAE : 69,818

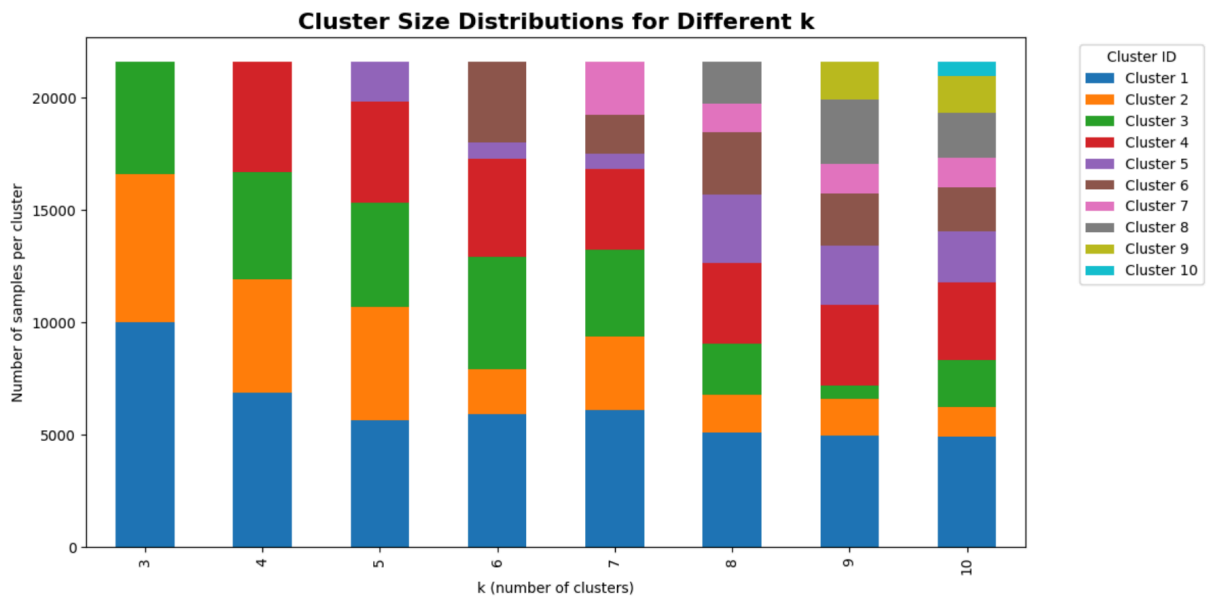
```

(Figure 3.3e) Evaluation Metrics (in Real Price Space) for Random Forest Model

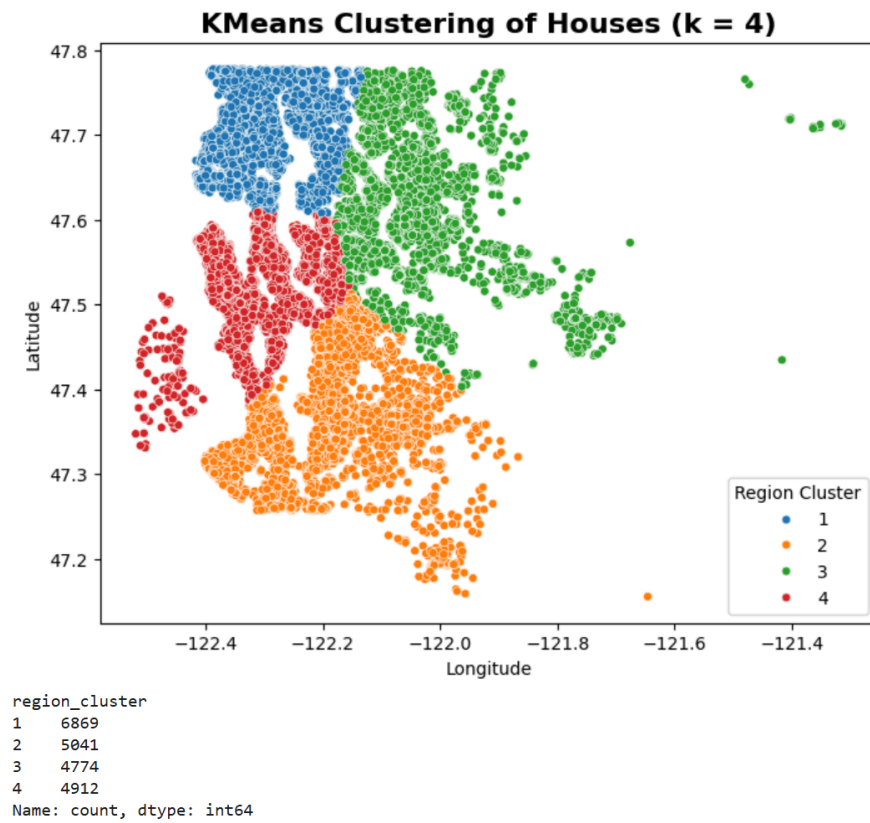
KMeans Clustering of Houses by Latitude & Longitude



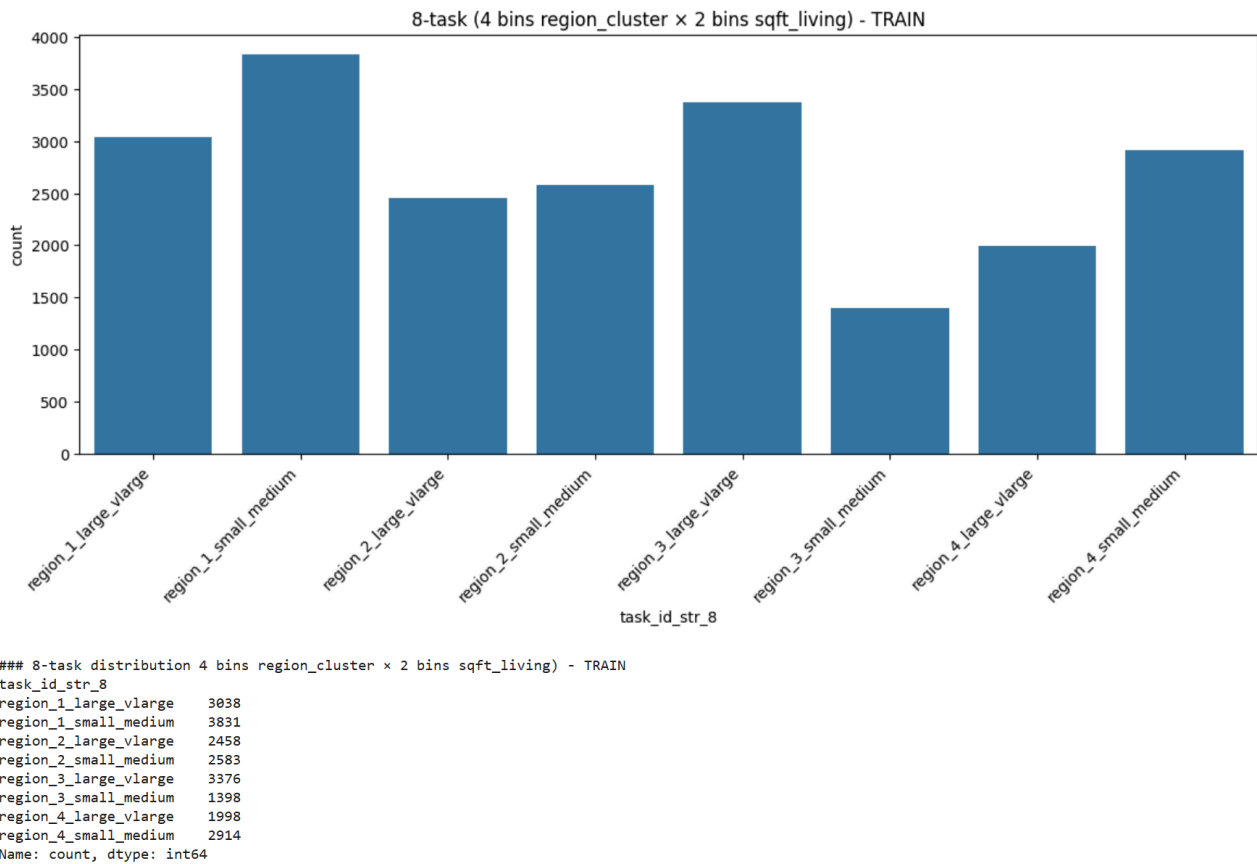
(Figure 3.4a) Cluster distributions for values $k = 3$ to $k = 10$ for visual cluster inspection



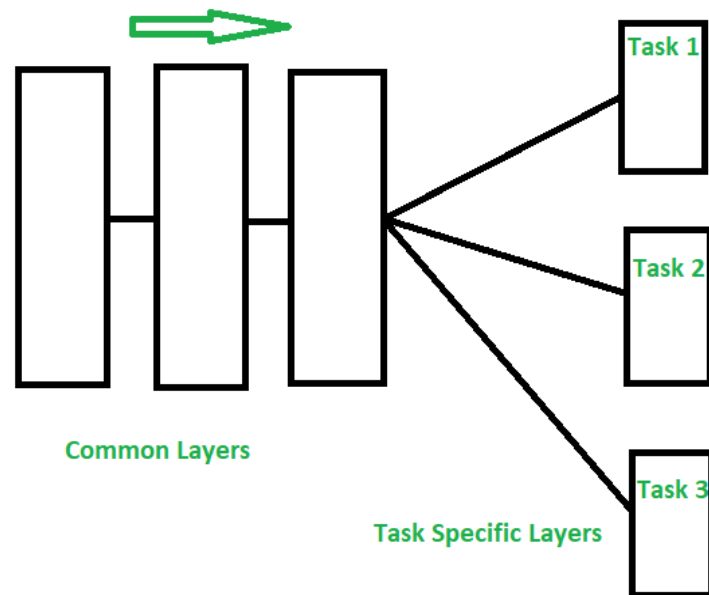
(Figure 3.4b) Cluster size distribution for values $k = 3$ to $k = 10$



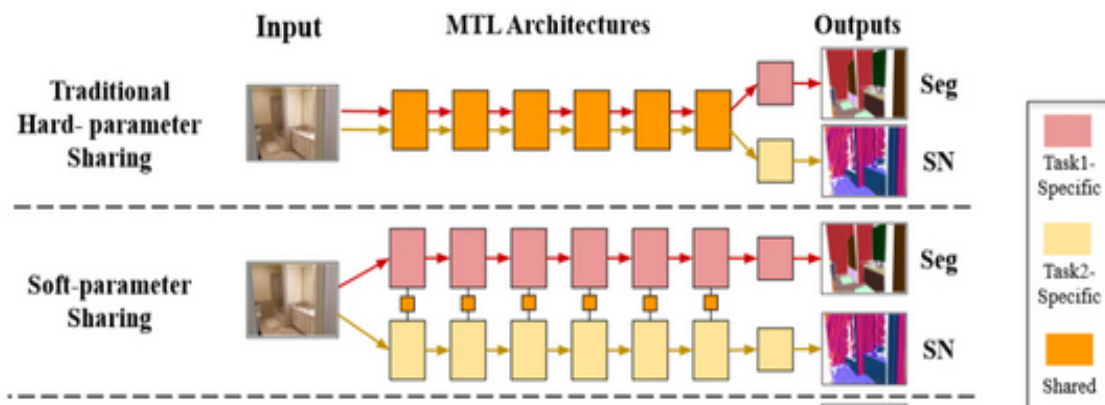
(Figure 3.4c) Cluster visualisation and cluster distribution for k = 4



(Figure 3.4d) Visualisation of task distribution and data counts per task after partitioning per task ID



(Figure 3.4e) Typical structure of MTL model (Srivastava, 2025)



(Figure 3.4f) Structures of a hard and soft-parameter sharing MTL model (Sun et al., 2020)


```

class LinearMTL(nn.Module):
    def __init__(self, D, T):
        super().__init__()
        # Deterministic initialization
        torch.manual_seed(42)
        self.W = nn.Parameter(torch.randn(D, T) * 0.01)

    def forward(self, X, task_id):
        W_sel = self.W[:, task_id]
        return (X.T * W_sel).sum(dim=0)

def l21_norm(W):
    return torch.norm(W, dim=1).sum()

# training function
def train_mtl(X, y, task_id, lambda_l21, steps=3000, lr=0.03, patience=250):
    model = LinearMTL(num_features, num_tasks)
    opt = optim.Adam([model.W], lr=lr, eps=1e-8)

    best_loss = float('inf')
    no_improve = 0

    for step in range(steps):
        opt.zero_grad()
        y_pred = model(X, task_id)

        loss = F.mse_loss(y_pred, y) + lambda_l21 * l21_norm(model.W)
        loss.backward()
        opt.step()

        if loss.item() < best_loss - 1e-7:
            best_loss = loss.item()
            best_W = model.W.data.clone()
            no_improve = 0
        else:
            no_improve += 1

        if no_improve >= patience:
            break

    model.W.data = best_W.clone()
    return model

```

```

# lambda cross validation
lambda_values = [1e-4, 5e-4, 1e-3]
kf = KFold(n_splits=5, shuffle=True, random_state=42)

val_scores = {}

for lam in lambda_values:
    fold_r2 = []

    for train_idx, val_idx in kf.split(X_train_mtl):
        Xtr_fold = torch.as_tensor(X_train_mtl.values[train_idx], dtype=torch.float32)
        ytr_fold = torch.as_tensor(y_train_mtl[train_idx].astype(np.float32))
        ttr_fold = torch.as_tensor(task_train_values[train_idx], dtype=torch.long)

        Xval_fold = torch.as_tensor(X_train_mtl.values[val_idx], dtype=torch.float32)
        yval_fold = torch.as_tensor(y_train_mtl[val_idx].astype(np.float32))
        tval_fold = torch.as_tensor(task_train_values[val_idx], dtype=torch.long)

        model = train_mtl(Xtr_fold, ytr_fold, ttr_fold, lambda_l21=lam, steps=2000, lr=0.03, patience=150)

        with torch.no_grad():
            pred_val = model(Xval_fold, tval_fold).cpu().numpy()

        fold_r2.append(r2_score(yval_fold.cpu().numpy(), pred_val))

    val_scores[lam] = np.mean(fold_r2)
    print(f"\n{lam:.1e} -> Mean CV R² = {val_scores[lam]:.4f}")

best_lambda = max(val_scores, key=val_scores.get)
print(f"\nBest λ selected: {best_lambda:.1e}")

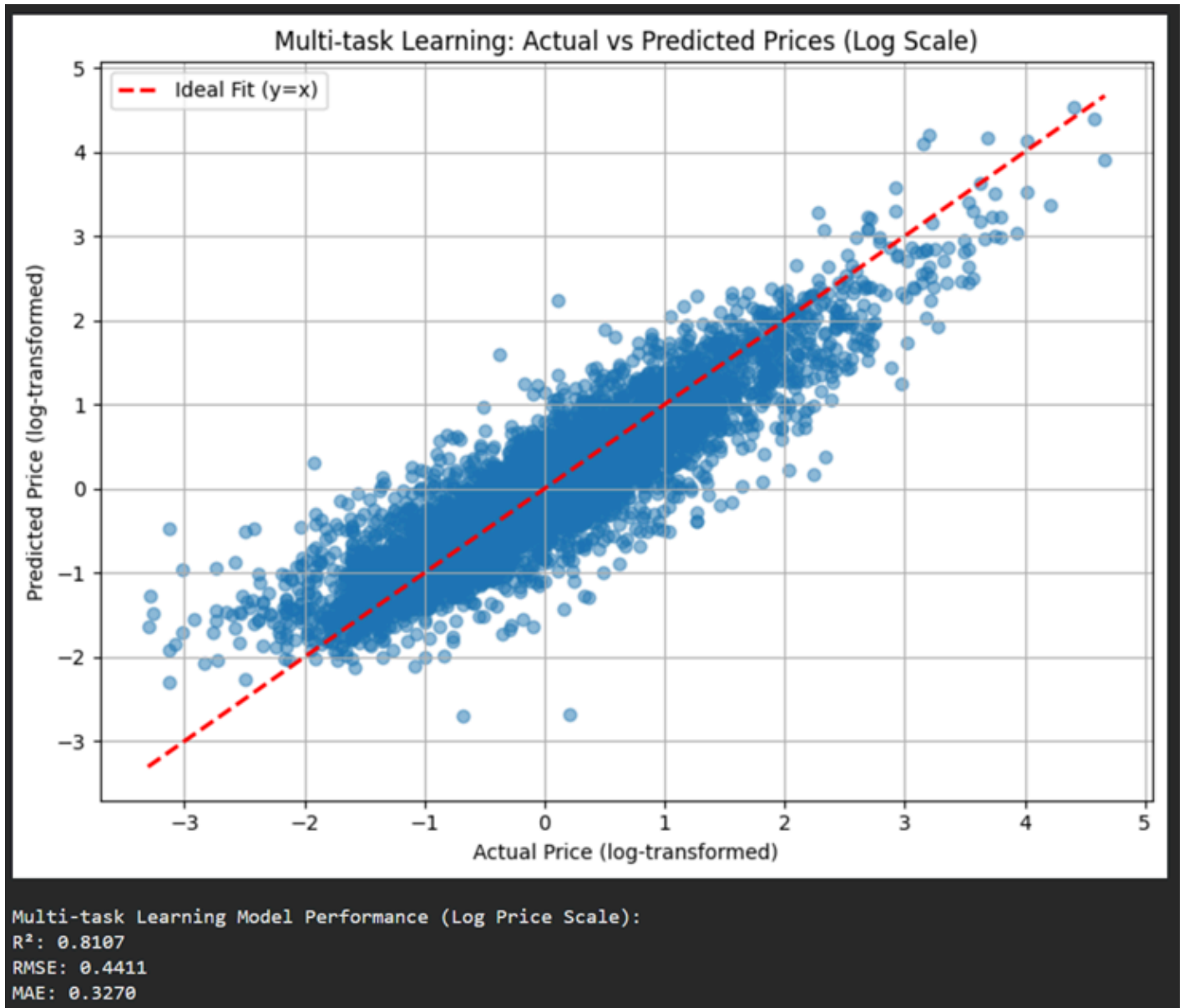
# final model
mtl_model = train_mtl(Xtr, ytr, ttr, lambda_l21=best_lambda, steps=3500, lr=0.03, patience=250)

```

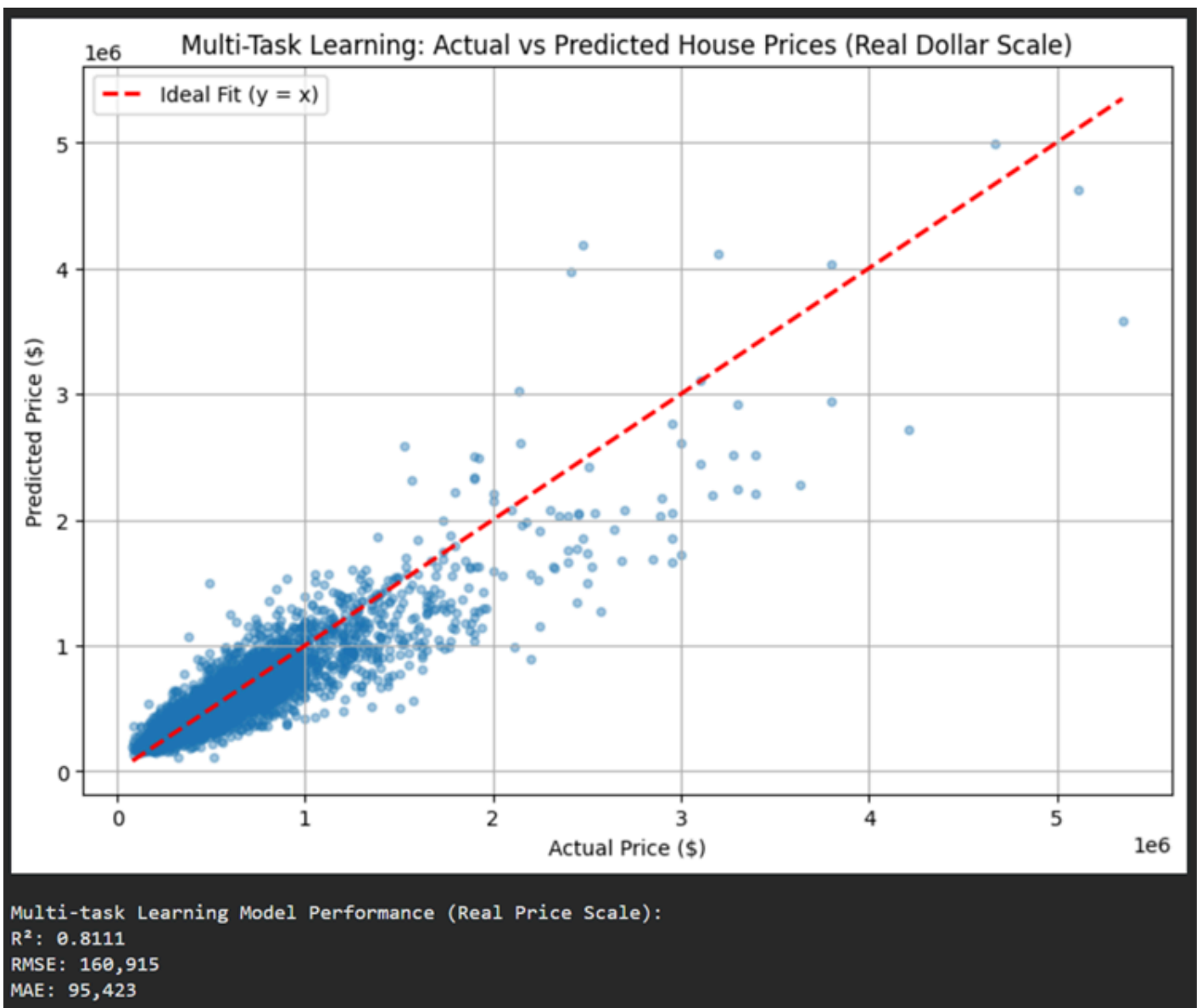
(Figure 3.4g) Full code of MTL architecture

```
 $\lambda=1.0e-04 \rightarrow \text{Mean CV } R^2 = 0.8177$   
 $\lambda=5.0e-04 \rightarrow \text{Mean CV } R^2 = 0.8189$   
 $\lambda=1.0e-03 \rightarrow \text{Mean CV } R^2 = 0.8183$   
  
Best  $\lambda$  selected:  $5.0e-04$ 
```

(Figure 3.4h) Best lambda value chosen after 5-fold cross-validation



(Figure 3.4i) Predicted vs Actual Prices (in Log Price Space) for Multi-Task Learning Model



(Figure 3.4j) Predicted vs Actual Prices (in Real Price Scale) for Multi-Task Learning Model